Kalman Filter Theory and Applications
State Space Concepts

https://github.com/musicarroll/kalman_course

Michael L. Carroll

October 16, 2024

©2023 by Michael L. Carroll

Part III

State Space Concepts

Topics

Part III

State Space Concepts

Topics

- Understanding the Equations: Heuristic Introduction

Part III

State Space Concepts

Topics

- Understanding the Equations: Heuristic Introduction
- Equation Drilldown: Taking the Equations Apart

- Understanding the Equations: Heuristic Introduction
- Equation Drilldown: Taking the Equations Apart
- **State Space Concepts** ($\geq$8 videos)

- Understanding the Equations: Heuristic Introduction
- Equation Drilldown: Taking the Equations Apart
- **State Space Concepts** ($\geq$8 videos)

State Space Concepts

Topics

State Space Concepts

Topics

- Dynamic Systems: The Concept of State Space

State Space Concepts

Topics

- Dynamic Systems: The Concept of State Space
- Deriving State Models from Differential Equations

State Space Concepts

Topics

- Dynamic Systems: The Concept of State Space
- Deriving State Models from Differential Equations
  - Examples of Differential Equations

- Dynamic Systems: The Concept of State Space
- Deriving State Models from Differential Equations
  - Examples of Differential Equations
- **Deriving State Models from Difference Equations**

- Dynamic Systems: The Concept of State Space
- Deriving State Models from Differential Equations
  - Examples of Differential Equations
- **Deriving State Models from Difference Equations**
  - Examples of Difference Equations

State Space Concepts

Topics

- Dynamic Systems: The Concept of State Space
- Deriving State Models from Differential Equations
  - Examples of Differential Equations
- **Deriving State Models from Difference Equations**
  - Examples of Difference Equations
- The State Transition Matrix

State Space Concepts

Topics

- Dynamic Systems: The Concept of State Space
- Deriving State Models from Differential Equations
  - Examples of Differential Equations
- **Deriving State Models from Difference Equations**
  - Examples of Difference Equations
- The State Transition Matrix
- State Space Modeling Diagrams: Block Diagrams, Signal Flow Graphs

- Dynamic Systems: The Concept of State Space
- Deriving State Models from Differential Equations
  - Examples of Differential Equations
- **Deriving State Models from Difference Equations**
  - Examples of Difference Equations
- The State Transition Matrix
- State Space Modeling Diagrams: Block Diagrams, Signal Flow Graphs
- State Space Modeling in Python

- Dynamic Systems: The Concept of State Space
- Deriving State Models from Differential Equations
  - Examples of Differential Equations
- **Deriving State Models from Difference Equations**
  - Examples of Difference Equations
- The State Transition Matrix
- State Space Modeling Diagrams: Block Diagrams, Signal Flow Graphs
- State Space Modeling in Python

Difference Equations

Overview

Difference Equations

Overview

- Definition and Importance

Difference Equations

Overview

- Definition and Importance
- Examples and Applications

Difference Equations

Overview

- Definition and Importance
- Examples and Applications
  - Control Systems
  - Signal Processing
  - Finance

Difference Equations

Overview

- Definition and Importance
- Examples and Applications
  - Control Systems
  - Signal Processing
  - Finance
- Relationship to Differential Equations

Difference Equations

Overview

- Definition and Importance
- Examples and Applications
  - Control Systems
  - Signal Processing
  - Finance
- Relationship to Differential Equations

Difference Equations

Importance and Relevance

- **Modeling Dynamic Systems:**
  - Captures changes between discrete time steps.
  - Useful in systems where variables evolve over time (e.g., population growth, economic indicators).

Difference Equations

Importance and Relevance

- **Modeling Dynamic Systems:**
  - Captures changes between discrete time steps.
  - Useful in systems where variables evolve over time (e.g., population growth, economic indicators).
- **Bridge to Differential Equations:**
  - Difference equations are the discrete-time counterparts to differential equations.
  - Used when systems are sampled at discrete intervals or in digital settings.

Difference Equations

Importance and Relevance

- **Modeling Dynamic Systems:**
  - Captures changes between discrete time steps.
  - Useful in systems where variables evolve over time (e.g., population growth, economic indicators).
- **Bridge to Differential Equations:**
  - Difference equations are the discrete-time counterparts to differential equations.
  - Used when systems are sampled at discrete intervals or in digital settings.

Applications of Difference Equations

Key Areas of Use

Applications of Difference Equations

Key Areas of Use

- **Control Systems:**
  - Feedback loops, system stability, and controller design.

Applications of Difference Equations

Key Areas of Use

- **Control Systems:**
  - Feedback loops, system stability, and controller design.
- **Signal Processing:**
  - Filtering and analysis of discrete-time signals.

Applications of Difference Equations

Key Areas of Use

- **Control Systems:**
  - Feedback loops, system stability, and controller design.
- **Signal Processing:**
  - Filtering and analysis of discrete-time signals.
- **Finance:**
  - Modeling investments, loans, and asset dynamics over time.

Applications of Difference Equations

Key Areas of Use

- **Control Systems:**
  - Feedback loops, system stability, and controller design.
- **Signal Processing:**
  - Filtering and analysis of discrete-time signals.
- **Finance:**
  - Modeling investments, loans, and asset dynamics over time.
- **Epidemiology:**
  - Predicting disease spread using discrete time models.

Applications of Difference Equations

Key Areas of Use

- **Control Systems:**
  - Feedback loops, system stability, and controller design.
- **Signal Processing:**
  - Filtering and analysis of discrete-time signals.
- **Finance:**
  - Modeling investments, loans, and asset dynamics over time.
- **Epidemiology:**
  - Predicting disease spread using discrete time models.

Difference Equations

Definitions

Difference Equations

Definitions

- **Scalar Difference Equation:** An equation involving differences between successive values of a scalar sequence.
  - Example of a scalar difference equation:

    $$y(n+1) = a(1)y(n) + a(2)y(n-1) + \cdots + a(k)y(n-k+1) + b$$

  - $y(n)$ is a scalar output at time step n, and $a(i)$ are scalar coefficients.

Difference Equations

Definitions

- **Scalar Difference Equation:** An equation involving differences between successive values of a scalar sequence.
  - Example of a scalar difference equation:

    $$y(n+1) = a(1)y(n) + a(2)y(n-1) + \cdots + a(k)y(n-k+1) + b$$

  - $y(n)$ is a scalar output at time step n, and $a(i)$ are scalar coefficients.
- **Vector Difference Equation:** When the output $y(n)$ is a vector, the difference equation becomes:

$$y_i(n+1) = \sum_{j=1}^{k} a_{ij} y_j(n) + b_i$$

  - $y_i(n)$ is the i-th component of the output vector at time step n.
  - a_{ij} are the matrix coefficients, and b_i are constants (or functions of time).

Difference Equations

Definitions

- **Scalar Difference Equation:** An equation involving differences between successive values of a scalar sequence.
  - Example of a scalar difference equation:

    $$y(n+1) = a(1)y(n) + a(2)y(n-1) + \cdots + a(k)y(n-k+1) + b$$

  - $y(n)$ is a scalar output at time step n, and $a(i)$ are scalar coefficients.
- **Vector Difference Equation:** When the output $y(n)$ is a vector, the difference equation becomes:

$$y_i(n+1) = \sum_{j=1}^{k} a_{ij} y_j(n) + b_i$$

  - $y_i(n)$ is the i-th component of the output vector at time step n.
  - a_{ij} are the matrix coefficients, and b_i are constants (or functions of time).
- The scalar equation is a special case of the vector equation when $y(n)$ is one-dimensional.

Difference Equations

Definitions

- **Scalar Difference Equation:** An equation involving differences between successive values of a scalar sequence.
  - Example of a scalar difference equation:

    $$y(n+1) = a(1)y(n) + a(2)y(n-1) + \cdots + a(k)y(n-k+1) + b$$

  - $y(n)$ is a scalar output at time step n, and $a(i)$ are scalar coefficients.
- **Vector Difference Equation:** When the output $y(n)$ is a vector, the difference equation becomes:

$$y_i(n+1) = \sum_{j=1}^{k} a_{ij} y_j(n) + b_i$$

  - $y_i(n)$ is the i-th component of the output vector at time step n.
  - a_{ij} are the matrix coefficients, and b_i are constants (or functions of time).
- The scalar equation is a special case of the vector equation when $y(n)$ is one-dimensional.

Higher-Order vs. First-Order Difference Equations

Scalar vs. Vector Representation

Higher-Order vs. First-Order Difference Equations

Scalar vs. Vector Representation

- **Higher-Order Scalar Difference Equation:**
  - Example: $y(n+1) = a(1)y(n) + a(2)y(n-1) + b$
  - This equation depends on multiple past values of $y(n)$, such as $y(n)$, $y(n-1)$, etc.

Higher-Order vs. First-Order Difference Equations

Scalar vs. Vector Representation

- **Higher-Order Scalar Difference Equation:**
  - Example: $y(n+1) = a(1)y(n) + a(2)y(n-1) + b$
  - This equation depends on multiple past values of $y(n)$, such as $y(n)$, $y(n-1)$, etc.
- **First-Order Vector Difference Equation:**
  - First-order equations involve only the current and next time steps.
  - Example: $x(n+1) = Ax(n) + Bu(n)$
  - Higher-order scalar equations can be converted into first-order vector equations.

Higher-Order vs. First-Order Difference Equations

Scalar vs. Vector Representation

- **Higher-Order Scalar Difference Equation:**
  - Example: $y(n+1) = a(1)y(n) + a(2)y(n-1) + b$
  - This equation depends on multiple past values of $y(n)$, such as $y(n)$, $y(n-1)$, etc.
- **First-Order Vector Difference Equation:**
  - First-order equations involve only the current and next time steps.
  - Example: $x(n+1) = Ax(n) + Bu(n)$
  - Higher-order scalar equations can be converted into first-order vector equations.

Converting Higher-Order Scalar to First-Order Vector Equations

Step 1: Introducing State Variables

Converting Higher-Order Scalar to First-Order Vector Equations

Step 1: Introducing State Variables

- **Example:** Start with a second-order scalar equation:

$$y(n+1) = a(1)y(n) + a(2)y(n-1) + b$$

Converting Higher-Order Scalar to First-Order Vector Equations

Step 1: Introducing State Variables

- **Example:** Start with a second-order scalar equation:

$$y(n+1) = a(1)y(n) + a(2)y(n-1) + b$$

- **Step 1:** Introduce state variables to represent the past time steps.

$$x_1(n) = y(n), \quad x_2(n) = y(n-1)$$

These state variables capture the dynamics of $y(n)$ and its past value.

Converting Higher-Order Scalar to First-Order Vector Equations

Step 1: Introducing State Variables

- **Example:** Start with a second-order scalar equation:

$$y(n+1) = a(1)y(n) + a(2)y(n-1) + b$$

- **Step 1:** Introduce state variables to represent the past time steps.

$$x_1(n) = y(n), \quad x_2(n) = y(n-1)$$

These state variables capture the dynamics of $y(n)$ and its past value.

Converting Higher-Order Scalar to First-Order Vector Equations

Step 2: State-Space Representation and Non-Linear Equations

Converting Higher-Order Scalar to First-Order Vector Equations

Step 2: State-Space Representation and Non-Linear Equations

- **Step 2:** Write the state-space equations:

$$x_1(n+1) = a(1)x_1(n) + a(2)x_2(n) + b$$

$$x_2(n+1) = x_1(n)$$

This converts the second-order scalar equation into a system of first-order equations.

Converting Higher-Order Scalar to First-Order Vector Equations

Step 2: State-Space Representation and Non-Linear Equations

- **Step 2:** Write the state-space equations:

$$x_1(n+1) = a(1)x_1(n) + a(2)x_2(n) + b$$

$$x_2(n+1) = x_1(n)$$

This converts the second-order scalar equation into a system of first-order equations.

- The system can now be represented as a vector equation:

$$x(n+1) = Ax(n) + Bu(n)$$

Converting Higher-Order Scalar to First-Order Vector Equations

Step 2: State-Space Representation and Non-Linear Equations

- **Step 2:** Write the state-space equations:

$$x_1(n+1) = a(1)x_1(n) + a(2)x_2(n) + b$$

$$x_2(n+1) = x_1(n)$$

  This converts the second-order scalar equation into a system of first-order equations.

- The system can now be represented as a vector equation:

$$x(n+1) = Ax(n) + Bu(n)$$

- **Non-linear Difference Equation:** Involves non-linear functions of the sequence values or the time indices.

Converting Higher-Order Scalar to First-Order Vector Equations

Step 2: State-Space Representation and Non-Linear Equations

- **Step 2:** Write the state-space equations:

$$x_1(n+1) = a(1)x_1(n) + a(2)x_2(n) + b$$

$$x_2(n+1) = x_1(n)$$

  This converts the second-order scalar equation into a system of first-order equations.

- The system can now be represented as a vector equation:

$$x(n+1) = Ax(n) + Bu(n)$$

- **Non-linear Difference Equation:** Involves non-linear functions of the sequence values or the time indices.

Difference Equations

State Variables and Outputs (Vector Nature)

- **State Variables:** $x(n)$
  - $x(n)$ is a **state vector** representing the system's internal state at time step n.
  - Dimension: $x(n) \in \mathbb{R}^d$ (where d is the number of states).

Difference Equations

State Variables and Outputs (Vector Nature)

- **State Variables:** $x(n)$
  - $x(n)$ is a **state vector** representing the system's internal state at time step n.
  - Dimension: $x(n) \in \mathbb{R}^d$ (where d is the number of states).
- **Outputs:** $y(n)$
  - $y(n)$ is the **output vector**, representing the observed outputs of the system at time step n.
  - Dimension: $y(n) \in \mathbb{R}^m$ (where m is the number of outputs).

Difference Equations

State Variables and Outputs (Vector Nature)

- **State Variables:** $x(n)$
  - $x(n)$ is a **state vector** representing the system's internal state at time step n.
  - Dimension: $x(n) \in \mathbb{R}^d$ (where d is the number of states).
- **Outputs:** $y(n)$
  - $y(n)$ is the **output vector**, representing the observed outputs of the system at time step n.
  - Dimension: $y(n) \in \mathbb{R}^m$ (where m is the number of outputs).

State-Space Output Equation

Relationship Between State and Output

State-Space Output Equation

Relationship Between State and Output

- The output $y(n)$ is related to the state $x(n)$ by the matrix C:

$$y(n) = Cx(n)$$

State-Space Output Equation

Relationship Between State and Output

- The output $y(n)$ is related to the state $x(n)$ by the matrix C:

$$y(n) = Cx(n)$$

- C is an $m \times d$ matrix, where:
  - m is the number of outputs.
  - d is the number of states.

State-Space Output Equation

Relationship Between State and Output

- The output $y(n)$ is related to the state $x(n)$ by the matrix C:

$$y(n) = Cx(n)$$

- C is an $m \times d$ matrix, where:
  - m is the number of outputs.
  - d is the number of states.

State Transition and Inputs

Control Inputs and Noise (Vector Nature)

State Transition and Inputs

Control Inputs and Noise (Vector Nature)

- **Control Inputs:** $u(n)$
  - $u(n)$ is the **input vector**, representing external inputs at time step n.
  - Dimension: $u(n) \in \mathbb{R}^p$ (where p is the number of control inputs).

State Transition and Inputs

Control Inputs and Noise (Vector Nature)

- **Control Inputs:** $u(n)$
  - $u(n)$ is the **input vector**, representing external inputs at time step n.
  - Dimension: $u(n) \in \mathbb{R}^p$ (where p is the number of control inputs).
- **Noise Inputs:** $w(n)$
  - $w(n)$ is the **noise vector**, representing process noise at time step n.
  - Dimension: $w(n) \in \mathbb{R}^q$ (where q is the dimension of the noise).

State Transition and Inputs

Control Inputs and Noise (Vector Nature)

- **Control Inputs:** $u(n)$
  - $u(n)$ is the **input vector**, representing external inputs at time step n.
  - Dimension: $u(n) \in \mathbb{R}^p$ (where p is the number of control inputs).
- **Noise Inputs:** $w(n)$
  - $w(n)$ is the **noise vector**, representing process noise at time step n.
  - Dimension: $w(n) \in \mathbb{R}^q$ (where q is the dimension of the noise).
- The state transition equation is given by:

$$x(n+1) = Ax(n) + Bu(n) + Gw(n)$$

State Transition and Inputs

Control Inputs and Noise (Vector Nature)

- **Control Inputs:** $u(n)$
  - $u(n)$ is the **input vector**, representing external inputs at time step n.
  - Dimension: $u(n) \in \mathbb{R}^p$ (where p is the number of control inputs).
- **Noise Inputs:** $w(n)$
  - $w(n)$ is the **noise vector**, representing process noise at time step n.
  - Dimension: $w(n) \in \mathbb{R}^q$ (where q is the dimension of the noise).
- The state transition equation is given by:

$$x(n+1) = Ax(n) + Bu(n) + Gw(n)$$

Difference Equations

Defining the State for a Simple System

- Start with a simple linear difference equation:

$$y(n+1) = ay(n) + bu(n)$$

This models how the output $y(n)$ changes over time based on its previous value and input $u(n)$.

- Start with a simple linear difference equation:

$$y(n+1) = ay(n) + bu(n)$$

  This models how the output $y(n)$ changes over time based on its previous value and input $u(n)$.

- Define the state variable $x(n)$ to represent the internal state of the system:

$$x(n) = y(n)$$

  In this case, the state coincides with the output, but it's helpful to treat them as distinct for generalization.

- Start with a simple linear difference equation:

$$y(n+1) = ay(n) + bu(n)$$

This models how the output $y(n)$ changes over time based on its previous value and input $u(n)$.

- Define the state variable $x(n)$ to represent the internal state of the system:

$$x(n) = y(n)$$

In this case, the state coincides with the output, but it's helpful to treat them as distinct for generalization.

State-Space Representation

General Form

State-Space Representation

General Form

- Using $x(n)$ as the state vector, we can write the general form of the state-space representation:

$$x(n+1) = Ax(n) + Bu(n)$$

  - A is an $d \times d$ matrix, where d is the number of state variables.
  - B is an $d \times p$ matrix, where p is the number of inputs.

State-Space Representation

General Form

- Using $x(n)$ as the state vector, we can write the general form of the state-space representation:

$$x(n+1) = Ax(n) + Bu(n)$$

  - A is an $d \times d$ matrix, where d is the number of state variables.
  - B is an $d \times p$ matrix, where p is the number of inputs.
- The output $y(n)$ is related to the state $x(n)$ via the matrix C:

$$y(n) = Cx(n)$$

  - C is an $m \times d$ matrix, where m is the number of outputs.

State-Space Representation

General Form

- Using $x(n)$ as the state vector, we can write the general form of the state-space representation:

$$x(n+1) = Ax(n) + Bu(n)$$

  - A is an $d \times d$ matrix, where d is the number of state variables.
  - B is an $d \times p$ matrix, where p is the number of inputs.
- The output $y(n)$ is related to the state $x(n)$ via the matrix C:

$$y(n) = Cx(n)$$

  - C is an $m \times d$ matrix, where m is the number of outputs.
- In general, the system parameters A, B, and C are matrices with the following dimensions:
  - $A \in \mathbb{R}^{d \times d}$
  - $B \in \mathbb{R}^{d \times p}$
  - $C \in \mathbb{R}^{m \times d}$

State-Space Representation

General Form

- Using $x(n)$ as the state vector, we can write the general form of the state-space representation:

$$x(n+1) = Ax(n) + Bu(n)$$

  - A is an $d \times d$ matrix, where d is the number of state variables.
  - B is an $d \times p$ matrix, where p is the number of inputs.
- The output $y(n)$ is related to the state $x(n)$ via the matrix C:

$$y(n) = Cx(n)$$

  - C is an $m \times d$ matrix, where m is the number of outputs.
- In general, the system parameters A, B, and C are matrices with the following dimensions:
  - $A \in \mathbb{R}^{d \times d}$
  - $B \in \mathbb{R}^{d \times p}$
  - $C \in \mathbb{R}^{m \times d}$

ARMA Models

Autoregressive (AR) Component

- **Autoregressive (AR):**

$$y(n) = \mu_{AR} + \sum_{i=1}^{p} a(i)y(n-i) + w(n)$$

- **Autoregressive (AR):**

$$y(n) = \mu_{AR} + \sum_{i=1}^{p} a(i)y(n-i) + w(n)$$

  - μ_{AR} is a constant drift term in the AR component.
  - $a(i)$ are the AR coefficients.
  - $w(n)$ is white noise, representing process noise.

- **Autoregressive (AR):**

$$y(n) = \mu_{AR} + \sum_{i=1}^{p} a(i)y(n-i) + w(n)$$

  - μ_{AR} is a constant drift term in the AR component.
  - $a(i)$ are the AR coefficients.
  - $w(n)$ is white noise, representing process noise.

ARMA Models

Moving Average (MA) Component and ARMA Summary

- **Moving Average (MA):**

$$y(n) = \mu_{MA} + w(n) + \sum_{j=1}^{q} b(j)w(n-j)$$

- **Moving Average (MA):**

$$y(n) = \mu_{MA} + w(n) + \sum_{j=1}^{q} b(j)w(n-j)$$

  - μ_{MA} is a constant drift term in the MA component.
  - $b(j)$ are the MA coefficients.
  - $w(n)$ is white noise, used as an input.

- **Moving Average (MA):**

$$y(n) = \mu_{MA} + w(n) + \sum_{j=1}^{q} b(j)w(n-j)$$

  - μ_{MA} is a constant drift term in the MA component.
  - $b(j)$ are the MA coefficients.
  - $w(n)$ is white noise, used as an input.

- **ARMA(p, q):**

$$y(n) = \mu + \sum_{i=1}^{p} a(i)y(n-i) + w(n) + \sum_{j=1}^{q} b(j)w(n-j)$$

where $\mu = \mu_{AR} + \mu_{MA}$.

- **Moving Average (MA):**

$$y(n) = \mu_{MA} + w(n) + \sum_{j=1}^{q} b(j)w(n-j)$$

  - μ_{MA} is a constant drift term in the MA component.
  - $b(j)$ are the MA coefficients.
  - $w(n)$ is white noise, used as an input.

- **ARMA(p, q):**

$$y(n) = \mu + \sum_{i=1}^{p} a(i)y(n-i) + w(n) + \sum_{j=1}^{q} b(j)w(n-j)$$

where $\mu = \mu_{AR} + \mu_{MA}$.

Explanation of μ in ARMA Models

Role of the Constant Term μ_{AR} in AR

Explanation of μ in ARMA Models

Role of the Constant Term μ_{AR} in AR

- μ_{AR} represents the drift term in the AR component of the ARMA model.

Explanation of μ in ARMA Models

Role of the Constant Term μ_{AR} in AR

- μ_{AR} represents the drift term in the AR component of the ARMA model.
- The AR component can be written as:

$$y(n) = \mu_{AR} + \sum_{i=1}^{p} a(i)y(n-i) + w(n)$$

This version ensures that the drift term μ_{AR} is applied correctly.

Explanation of μ in ARMA Models

Role of the Constant Term μ_{AR} in AR

- μ_{AR} represents the drift term in the AR component of the ARMA model.
- The AR component can be written as:

$$y(n) = \mu_{AR} + \sum_{i=1}^{p} a(i)y(n-i) + w(n)$$

This version ensures that the drift term μ_{AR} is applied correctly.

Explanation of μ in ARMA Models

Role of the Constant Term μ_{MA} in MA

Explanation of μ in ARMA Models

Role of the Constant Term μ_{MA} in MA

- In the MA part, μ_{MA} shifts the series, adjusting the overall level.

$$y(n) = \mu_{MA} + w(n) + \sum_{j=1}^{q} b(j)w(n-j)$$

Explanation of μ in ARMA Models

Role of the Constant Term μ_{MA} in MA

- In the MA part, μ_{MA} shifts the series, adjusting the overall level.

$$y(n) = \mu_{MA} + w(n) + \sum_{j=1}^{q} b(j)w(n-j)$$

- μ_{MA} represents the long-term mean around which the series fluctuates.

Explanation of μ in ARMA Models

Role of the Constant Term μ_{MA} in MA

- In the MA part, μ_{MA} shifts the series, adjusting the overall level.

$$y(n) = \mu_{MA} + w(n) + \sum_{j=1}^{q} b(j)w(n-j)$$

- μ_{MA} represents the long-term mean around which the series fluctuates.
- μ_{MA} is not the mean of the noise $w(n)$ (which is zero), but a constant drift.

Explanation of μ in ARMA Models

Role of the Constant Term μ_{MA} in MA

- In the MA part, μ_{MA} shifts the series, adjusting the overall level.

$$y(n) = \mu_{MA} + w(n) + \sum_{j=1}^{q} b(j)w(n-j)$$

- μ_{MA} represents the long-term mean around which the series fluctuates.
- μ_{MA} is not the mean of the noise $w(n)$ (which is zero), but a constant drift.

Summary

Vector Check

Summary

Vector Check

- Where are we?

Summary

Vector Check

- Where are we?
  - Defined linear difference equations

Summary

Vector Check

- Where are we?
  - Defined linear difference equations
  - Showed how to treat these in terms of state space concepts

Summary

Vector Check

- Where are we?
  - Defined linear difference equations
  - Showed how to treat these in terms of state space concepts
- What's next?

Summary

Vector Check

- Where are we?
  - Defined linear difference equations
  - Showed how to treat these in terms of state space concepts
- What's next?
  - Some concrete examples

Summary

Vector Check

- Where are we?
  - Defined linear difference equations
  - Showed how to treat these in terms of state space concepts
- What's next?
  - Some concrete examples
  - Reminder: Python scripts are in `https://github.com/musicarroll/kalman_course`

Summary

Vector Check

- Where are we?
  - Defined linear difference equations
  - Showed how to treat these in terms of state space concepts
- What's next?
  - Some concrete examples
  - Reminder: Python scripts are in `https://github.com/musicarroll/kalman_course`